

COAL MINE GOVERNANCE PLATFORM

Detailed Implementation Solution | Hackathon-Ready Architecture

Purpose: convert fragmented mine governance records into a controlled workflow for evidence capture, compliance tracking, risk prioritization, corrective action, and accountability.

CORE LOOP	FIELD TO MANAGEMENT
Detect -> Record -> Validate -> Assign -> Act -> Verify -> Close	PWA field capture -> API -> PostgreSQL -> Risk engine -> Dashboard -> Audit trail

Design constraint: the prototype must work with synthetic data and remain implementable by a 3 to 5 person student team. Production-only features are separated from the hackathon MVP.

The revised scope follows the uploaded senior architecture review. It reduces role count, replaces React Native with a PWA, treats the risk score as a configurable index rather than a predictive model, makes auditability a core feature, and pushes low-ROI features such as OCR and blockchain out of the critical path.

1. Document Map

#	Section
2	Problem interpretation and product boundary
3	Core users and role boundaries
4	MVP product scope and feature priority
5	Detailed functional design
6	End-to-end system architecture
7	Recommended technology stack
8	Data model and database design
9	API surface
10	AI, risk scoring and anomaly logic
11	PWA offline field capture
12	GIS and evidence integrity
13	OCR and document workflow
14	Notifications and escalation
15	Security and audit trail
16	Implementation phases and dependencies
17	Team allocation and effort
18	24-hour and 48-hour execution plans
19	Testing and acceptance criteria
20	Demo script and strongest USPs
21	Production evolution and explicit exclusions

2. Problem Interpretation and Product Boundary

What the platform is: a governance control system for mine-level evidence, compliance obligations, observations, corrective actions, and management review.

What the platform is not: a mining production control system, payroll system, sensor platform, legal database, or replacement for existing enterprise systems.

The implementation should concentrate on one measurable governance question: **Which mine issues need attention now, who owns them, what evidence supports the finding, and has the issue actually been closed?**

Problem in PS	Platform response	Primary evidence
Fragmented records	Single data model and dashboard	One record ID linked to mine, user, finding and action
Delayed field reporting	PWA inspection capture with GPS, time and photo	Inspection is stored locally and synced after reconnect
Compliance gaps	Due-date engine and status logic	Overdue status plus owner and deadline
Weak accountability	Corrective action state machine and audit timeline	Every assignment, escalation, evidence upload and verification is logged
Limited management visibility	Mine scorecard and risk index	Mine ranking plus top contributing factors
Duplicate or incomplete submissions	Server-side completeness gate and idempotent sync	422 validation response and temp_uuid deduplication
Paper and PDF dependency	Optional OCR draft creation	Extracted fields enter normal review workflow, not a separate silo

Scope rule: every proposed feature must improve one of the following measurable outcomes: shorter reporting delay, fewer overdue actions, higher evidence completeness, faster closure, clearer responsibility, or better risk prioritization. Features that do not improve one of these outcomes stay out of the MVP.

3. Core Users and Role Boundaries

Use three operational roles in the prototype. Keep other enterprise roles in the database as configuration only. This reduces UI, testing and demo complexity while preserving a path to production RBAC.

Role	Can do	Cannot do	Demo purpose
Inspector	Create inspections, attach photo, capture GPS, submit observations, view own assigned work	Verify or close own corrective action, edit another user's records	Shows field evidence capture
Mine Manager	View one mine, review inspections, assign actions, verify evidence, close actions, view risk factors	View unrelated mines, alter global weights unless given admin permission	Shows local governance workflow
Corporate Manager	View all mines, compare mine metrics, view risk ranking, configure risk weights in demo, review audit trail	Submit field inspections as another user	Shows enterprise prioritization
Admin config only	Role definitions, weight configuration and seed reset endpoint	No dedicated dashboard in MVP	Reserved for future expansion

RBAC acceptance test: an Inspector token calling a manager-only verification endpoint must receive HTTP 403. A Mine Manager token must be restricted to the manager's mine. A Corporate Manager must receive cross-mine dashboard data.

4. MVP Product Scope and Feature Priority

The revised MVP has five feature groups. Database schema, migrations, deterministic seed data and reset tooling are foundational tasks rather than separate product features.

Priority	Feature group	Why it stays here	Target effort
P0	Compliance register + due-date logic	Directly maps to the stated compliance problem	12-16 h
P0	Inspection capture + completeness gate	Creates trustworthy field evidence	10-14 h
P0	Corrective action state machine + audit log	Creates the accountability loop	12-16 h
P0	Dashboard + configurable risk index	Turns records into management decisions	14-18 h
P0	PWA offline capture + idempotent sync	Makes field workflow credible in poor connectivity	6-8 h
P1	GIS markers + severity filter	Adds geographic evidence without zone-polygon complexity	6-8 h
P1	Alerts + escalation	Makes overdue conditions actionable	4-6 h
P1	Recurring violation detection	Shows pattern recognition using deterministic logic	2-3 h
P1	Action accountability timeline + workload suggestion	Adds visible workflow depth	7-11 h
P2	OCR to draft record	Useful only when connected to workflow	8-12 h
P2	Comparative scorecard export	Useful presentation feature, not core governance	6-8 h
P2	Contractor compliance score	Useful after core workflow is stable	4-6 h
P2	Production anomaly via rolling z-score	Optional analytics depth	2-3 h

Explicit cuts: blockchain audit layer, custom chatbot, React Native, IoT ingestion, Kubernetes, microservices, seven-role UI, zone heatmaps, and LLM report summarization. The uploaded review identifies these as poor uses of hackathon capacity or unsupported by available data. [Senior architect review, scope recommendation]

5. Detailed Functional Design

5.1 Compliance register

Each compliance requirement contains: requirement_id, mine_id, category, title, frequency, due_date, responsible_role, responsible_user_id, status, evidence_document_id, source_reference, created_at and updated_at. Status is derived where possible instead of manually typed. Example logic: compliant, due_soon, overdue, not_applicable.

5.2 Inspection capture

The inspector records mine, location, observation category, description, severity, GPS latitude and longitude, GPS accuracy, timestamp, photo, and optional corrective-action suggestion. The API rejects incomplete submissions when configured rules fail. High and Critical observations require a photo in the demo configuration.

5.3 Corrective action state machine

Use explicit states: OPEN -> ASSIGNED -> IN_PROGRESS -> VERIFICATION -> CLOSED. REOPENED is an exception state. A state transition writes an audit event. Verification must be performed by a different authorized user than the action owner.

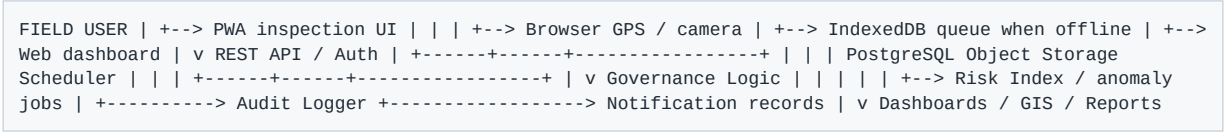
Transition	Allowed by	Required data	Audit event
OPEN -> ASSIGNED	Mine Manager	owner_id, due_date	ASSIGNED
ASSIGNED -> IN_PROGRESS	Owner	comment optional	STARTED
IN_PROGRESS -> VERIFICATION	Owner	evidence_id required	EVIDENCE_UPLOAD
VERIFICATION -> CLOSED	Mine Manager	verification decision	VERIFIED + CLOSED
Any open state -> REOPENED	Manager	reason	REOPENED
Overdue -> ESCALATED	Scheduler/System	escalation level	ESCALATED

5.4 Dashboard

Widget	Data source	Decision supported
Compliance rate	compliance_records	Which mines have weak compliance?
Open violations	violations	How much unresolved exposure exists?
Overdue actions	corrective_actions	Where is work aging?
Mine risk index	risk_scores or calculated view	Which mine needs attention first?
Risk reasons	risk components	Why is this mine ranked high?
Overdue aging	due_date vs current date	Are old actions accumulating?
Recent field observations	inspections/observations	What changed recently?

6. End-to-End System Architecture

Use a modular monolith, not microservices. The prototype needs one backend deployment and one relational database. Keep AI logic as a separate Python module or service boundary inside the same repository so it can be extracted later.



Key data flow rules:

- Every business object has a UUID and mine_id where relevant.
- Every state-changing request carries an authenticated actor ID into the audit logger.
- Photos and documents live in object storage. PostgreSQL stores metadata, hash and ownership.
- Long operations such as OCR and scheduled escalation run outside the synchronous request path.
- Offline inspections are create-only in the MVP. Use temp_uuid as the idempotency key to prevent duplicate uploads after reconnect.

7. Recommended Technology Stack

Layer	Choice	Implementation reason	Do not use in MVP
Web UI	React + Vite	Fast component development, simple deployment	Custom rendering engine
Field UI	React PWA	Runs in mobile Chrome, avoids native build pipeline	React Native
Backend	FastAPI or Node.js + Express	Simple REST endpoints and quick iteration	Microservice mesh
Database	PostgreSQL	Relational data, transactions, JSONB, strong constraints	NoSQL-first design
Offline	IndexedDB + Service Worker	Browser-native local persistence and app shell cache	Native SQLite wrapper
Charts	Recharts or Chart.js	Prebuilt charts, low implementation cost	D3 from scratch
GIS	Leaflet + OpenStreetMap tiles	Simple point mapping	Heavy GIS platform
OCR	PaddleOCR or Tesseract	Good enough for controlled sample documents	Cloud OCR dependency
Auth	JWT + bcrypt/argon2	Simple role-aware API protection	Complex identity federation
Storage	Local/S3-compatible object storage	Stores evidence files separately from transactional rows	Binary files in database
Scheduler	APScheduler/Celery or backend cron	Enough for reminders and risk recompute	Kafka
Deployment	Docker Compose, optional Render/Railway	Reproducible local demo with optional cloud	Kubernetes
Source control	GitHub	Team collaboration and issue board	Distributed multi-repo architecture

The review recommends PWA, IndexedDB, application-level audit inserts, deterministic UUID seeds, rolling z-score anomaly logic, Docker Compose and prebuilt chart libraries. [Senior architect review, technology recommendations]

8. Data Model and Database Design

Use a normalized relational core with explicit foreign keys and indexes. Avoid storing repeated dashboard metrics as hand-edited values. Derive them from transactional tables or refreshable summary queries.

Table	Key fields	Important indexes / rules
users	id, name, email, role_id, mine_id, password_hash, active	unique(email), index(mine_id, role_id)
roles	id, name	unique(name)
mines	id, name, subsidiary, latitude, longitude, active	index(subsidiary), geospatial index if PostGIS is used
compliance_requirements	id, category, title, frequency, source_reference	index(category)
compliance_records	id, mine_id, requirement_id, due_date, status, owner_id, evidence_id	index(mine_id, due_date), index(status)
inspections	id, mine_id, inspector_id, occurred_at, latitude, longitude, gps_accuracy_m, temp_uuid	unique(temp_uuid), index(mine_id, occurred_at)
observations	id, inspection_id, category, severity, description, photo_id	index(inspection_id, severity)
violations	id, observation_id, mine_id, category, severity, fingerprint	index(mine_id, category, fingerprint)
corrective_actions	id, violation_id, owner_id, due_date, status, created_at, closed_at	index(owner_id, status), index(mine_id, due_date, status)
audit_logs	id, entity_type, entity_id, actor_id, action, before_json, after_json, created_at	index(entity_type, entity_id, created_at)
documents	id, storage_key, sha256, mime_type, uploaded_by, status	unique(storage_key), index(sha256)
notifications	id, user_id, type, entity_id, created_at, read_at	index(user_id, read_at)
risk_scores	id, mine_id, score, components_json, reasons_json, calculated_at	index(score), unique(mine_id, calculated_at)

Audit log rule: never update or delete an audit row through normal application endpoints. The application adds a new event for every significant state change. This is the same high-impact, low-complexity approach identified in the uploaded review. [Senior architect review, audit recommendation]

9. API Surface

Keep endpoints focused on business operations. Avoid CRUD endpoints for every internal object when a workflow endpoint expresses the rule more clearly.

Method	Endpoint	Purpose	Role
POST	/auth/login	Issue JWT	All
GET	/dashboard/summary	Mine or enterprise summary	Manager/Corporate
GET	/dashboard/overdue-aging?mine_id=	Return overdue age buckets	Manager/Corporate
GET	/risk/mines	Return mines ranked by risk index	Corporate
GET	/risk/mines/{id}	Return score, components and reasons	Manager/Corporate
GET	/compliance?mine_id=	List requirements and current status	Manager/Corporate
POST	/compliance	Create or import compliance record	Manager
POST	/inspections	Create inspection; validate GPS/photo rules	Inspector
GET	/inspections/{id}	Inspection detail	Authorized users
GET	/gis/observations	Return map-ready points	Manager/Corporate
POST	/corrective-actions	Create and assign action	Manager
PATCH	/corrective-actions/{id}/status	Advance state	Owner/Manager
POST	/corrective-actions/{id}/verify	Verify evidence and close	Manager
GET	/corrective-actions/{id}/timeline	Return audit events	Manager/Corporate
GET	/users/workload?mine_id=	Show assignment workload	Manager
POST	/admin/reset-demo	Reset deterministic seed data	Admin

Example validation response for an incomplete inspection:

```
{"status":422,"missing_fields":["GPS_PRESENT","PHOTO_REQUIRED_FOR_HIGH_SEVERITY"],"message":"Inspection cannot be submitted until required evidence is attached."}
```

10. AI, Risk Scoring and Anomaly Logic

The prototype should not claim that the mine score is a trained predictive model. With synthetic data and no labeled incident outcome, the most defensible method is a configurable weighted index. The user-facing screen should explicitly call it a **Governance Risk Index**.

Component	Example normalization	Weight	Reason
Compliance pressure	100 - compliance_rate_pct	0.30	Higher non-compliance increases score
Violation severity	Weighted count by severity, normalized 0-100	0.25	High and Critical findings dominate
Corrective action aging	Normalized overdue action burden	0.20	Captures unresolved backlog
Safety exposure	Recent safety observation severity	0.15	Recent field evidence
Operational anomaly	Recent z-score flags	0.10	Adds operational context

Risk score = $\sum(\text{component_score} \times \text{weight})$. Each component is 0 to 100. Weights sum to 1.00. Store the component values and reasons so the UI can explain the result.

Example explanation: **Risk 78/100**. Main contributors: compliance rate 45%, three High severity findings still open, four actions overdue by more than 15 days. The score is a prioritization aid, not a claim that an incident will occur.

10.1 Recurring violation detection

Create a deterministic fingerprint such as category + mine_zone + contractor_id. Count occurrences in the last 90 days. Flag recurring when count ≥ 3 . Show the matched records so the user can inspect the evidence.

10.2 Production anomaly

Use a rolling 30-day z-score. Flag an anomaly when absolute $z > 2.0$, subject to a minimum history threshold such as 14 observations. With synthetic data, this is transparent and easy to test. The uploaded review specifically recommends rolling z-score over Isolation Forest for the prototype. [Senior architect review, anomaly recommendation]

10.3 Explainable trend text

Compare current risk components to the previous period and generate a deterministic sentence. Example: "Risk increased from 62 to 78 because corrective-action aging rose by 16 points." Do not require an LLM for this.

11. PWA Offline Field Capture

Use a create-only offline model. The browser stores draft inspection records and a sync queue. There is no offline editing conflict in the MVP because an inspection receives a permanent server ID only after successful synchronization.

```
ONLINE Inspector submits -> API -> DB -> 201 Created OFFLINE Inspector submits -> IndexedDB ->
queue[temp_uuid] -> UI shows "1 pending" RECONNECT Service worker / online event -> POST with temp_uuid ->
server checks unique temp_uuid -> new record: insert -> duplicate retry: return existing record -> UI removes
queue item
```

Local field	Purpose
temp_uuid	Idempotency key; UUID v4 on device
payload_json	Inspection data before server ID exists
photo_blob / photo_ref	Evidence attachment until upload completes
created_local_at	Device capture time
sync_status	pending, uploading, synced, failed
last_error	Human-readable retry reason

Offline acceptance test: disable network, submit one High-severity inspection with GPS and photo, show one pending item, restore connectivity, observe exactly one server record and zero duplicates. The review rates PWA plus IndexedDB as buildable and rejects native React Native for the hackathon. [Senior architect review, offline recommendation]

12. GIS and Evidence Integrity

GIS is P1 because the governance value is useful but the core accountability workflow must not depend on it. Store latitude, longitude and GPS accuracy with every field inspection. Render points using Leaflet. Start with mine markers and observation markers. Use severity filtering instead of zone heatmaps.

Map layer	Source	Demo behavior
Mine marker	mines.latitude/longitude	Click to open mine summary
Observation point	observations via inspections	Marker shows severity, time and inspector
Severity filter	observation.severity	Show only High/Critical, etc.
Recent observation list	inspection occurred_at	Selecting row centers the map
Evidence link	photo/document metadata	Open evidence associated with marker

Evidence integrity: when a photo is uploaded, compute SHA-256 and store the hash with storage key, uploader ID and timestamp. This creates an evidence fingerprint without requiring blockchain. The integrity check should answer: **Is this file the same file that was originally attached?**

13. OCR and Document Workflow

OCR is P2, not a disconnected upload demo. When implemented, the result must become a draft normal record.

PDF/Image -> OCR -> extracted text -> field extraction -> draft compliance/observation -> human review -> confirm -> normal database record -> dashboard

Step	Implementation
OCR	PaddleOCR or Tesseract on clean, typed sample documents
Field extraction	Regex or template matching for mine, date, requirement, violation, due date
Validation	Show extracted values plus "Needs review" when expected fields do not match
Save	On confirmation, create draft compliance_record or observation with source_document_id
Audit	Record who confirmed extracted data and when

The uploaded review correctly identifies that OCR is weak as a standalone demo and becomes meaningful only when its output enters the governance workflow. [Senior architect review, OCR recommendation]

14. Notifications and Escalation

Do not build a multi-channel notification platform for the hackathon. Create notification records in the database and display them in-app. Add email as an optional extension if time allows.

Trigger	Action	Escalation rule
Due in 3 days	Create reminder for owner	No escalation
Due today	Create high-priority reminder	Mine Manager notified
7 days overdue	Create escalation record	Corporate Manager notified
Critical observation created	Create immediate notification	Mine Manager alerted
Action reopened	Notify original owner + manager	No automatic closure

Scheduler implementation: run every hour or on-demand in demo mode. To avoid duplicate alerts, use a unique key such as trigger_type + entity_id + trigger_date in notifications.

15. Security and Audit Trail

Hackathon security should demonstrate correct authorization and evidence traceability. Production security would add enterprise identity, centralized secrets, managed object storage, key rotation, network controls, monitoring and disaster recovery.

Control	MVP implementation	Production extension
Authentication	JWT, password hash, token expiry	SSO/OIDC, MFA
Authorization	Three-role RBAC plus mine_id scoping	Policy engine and enterprise identity
Input validation	Pydantic/Zod/schema validation	Central API gateway validation
SQL injection	ORM/parameterized queries	Same plus WAF and security testing
File upload	Allowlist MIME, size limit, SHA-256, stored outside DB	Malware scanning, isolated bucket
Audit	Application-level append-only events	Central immutable audit storage
Secrets	Environment variables	Managed secret vault
Backups	Database dump for demo	Automated encrypted backups + recovery drills

Audit viewer fields: timestamp, actor, entity type, entity ID, action, before JSON, after JSON, detail. The high-value demo feature is the corrective-action timeline, which answers who handled the action, when ownership changed, when evidence appeared, and when closure was verified.

16. Implementation Phases and Dependencies

Phase	Core tasks	Deliverable	Exit criterion
0. Scope lock	Freeze P0, define seed scenarios, repo setup	README, issue board, architecture decision record	Every P0 task has owner and dependency
1. Data foundation	Migrations, tables, deterministic seed, reset	Working PostgreSQL + seed_reset.py	Reset completes under 30 sec
2. Auth/RBAC	Login, token middleware, role/mine checks	Protected API	Unauthorized calls return 401/403
3. Compliance	CRUD, status derivation, overdue logic	Compliance API + UI	Overdue record appears correctly
4. Inspection	Form, GPS, photo, completeness gate	Inspection API + UI	High severity requires evidence
5. Corrective action	State machine, evidence, verify, audit events	Action queue + timeline data	Cannot self-verify
6. Dashboard/Risk	Summary queries, score index, reasons, trend	Mine and enterprise dashboard	Values match DB fixtures
7. Offline PWA	IndexedDB queue, sync, temp_uuid dedupe	Offline inspection path	One inspection creates one server record
8. P1 depth	GIS, alerts, recurrence, workload	Polished demo depth	No P0 regression
9. Finalize	Testing, deployment, demo reset, rehearsal	Frozen demo build	Full path works without DB edits

Critical path: schema -> auth -> compliance/inspection -> corrective action + audit -> dashboard -> risk -> offline sync.

Parallel tracks: frontend shell, seed data, risk calculation and GIS can begin as soon as interfaces are agreed. The uploaded review identifies the same critical dependencies and parallelizable tracks. [Senior architect review, dependency recommendation]

17. Team Allocation and Effort

Use a four-person team as the reference model. If only three members are available, combine roles 2 and 3.

Member	Primary ownership	Secondary ownership	Main outputs
1. Backend/Data	PostgreSQL, FastAPI/Node, auth, compliance, actions	Seed/reset	Schema, APIs, migrations, audit logger
2. Frontend	React dashboard, tables, charts	API integration	Login, dashboard, action queue
3. Field/GIS	PWA inspection, IndexedDB, Leaflet	Evidence upload	Mobile inspection, offline sync, map
4. AI/Integration	Risk index, trend logic, recurrence	QA, demo data	Risk engine, explanations, integration testing

Workstream	Estimated person-hours	Priority
Data + seed + reset	10-14	P0
Auth/RBAC	8-10	P0
Compliance	12-16	P0
Inspection + evidence	10-14	P0
Corrective action + audit	12-16	P0
Dashboard + risk	14-18	P0
Offline PWA	6-8	P0
GIS	6-8	P1
Alerts + recurrence	6-9	P1
Timeline + workload	7-11	P1
OCR	8-12	P2
Testing + deployment + demo	12-18	P0/P1

Target P0 engineering budget: approximately 72-96 person-hours, excluding final testing and demo rehearsal. This is intentionally below the earlier 202-288 hour estimate because the review identified scope and platform choices that do not fit a 48-hour student build. [Senior architect review, effort concern]

18. 24-Hour and 48-Hour Execution Plans

24-hour target: produce a reliable web-only version of the complete governance loop. Offline PWA can be added only after the core path works.

0-3 h: repo, schema skeleton, seed fixtures 2-5 h: auth + RBAC 4-8 h: compliance + inspection APIs 6-11 h: frontend shell + dashboard cards 9-14 h: corrective action state machine + audit 12-17 h: dashboard integration + risk index 16-20 h: verification flow + demo data tuning 20-24 h: end-to-end test, deploy, rehearse

48-hour target: add offline PWA, GIS, and one supporting USP after P0 is frozen.

Hours 0-20: P0 implementation 20-26: P0 integration and reset test 24-32: PWA offline queue + sync 28-36: GIS markers + severity filter 32-38: recurrence + alerts 36-42: accountability timeline + workload suggestion 40-44: security and regression tests 44-46: demo rehearsal 46-48: final reset, deployment freeze

Stop rule: at the moment P0 fails any acceptance criterion, stop adding features. Fix the blocking path first.

19. Testing and Acceptance Criteria

Test the governance rules, not just screens. The most important failures are unauthorized verification, duplicate offline submissions, incomplete evidence, incorrect risk explanations, and broken state transitions.

Test	Expected result
Inspector login	JWT returned; manager endpoints return 403
Mine scope	Mine Manager cannot access another mine
Inspection without GPS	HTTP 422 with GPS_PRESENT failure
High severity without photo	HTTP 422 with PHOTO_REQUIRED_FOR_HIGH_SEVERITY
Offline inspection retry	Only one server record created for one temp_uuid
Create corrective action	Status is ASSIGNED and audit event exists
Owner self-verification	HTTP 403
Different manager verifies	Status changes to CLOSED and audit events exist
Risk ranking	Highest seeded score is first
Risk reasons	At least three reasons match seeded conditions
Overdue aging	Actions appear in correct age bucket
Seed reset	Same deterministic data returns after reset
Audit timeline	Events show actor and timestamp in chronological order
GIS	Observation appears at recorded coordinates
Regression	Full demo runs without manual SQL edits

Full demo acceptance criteria from the uploaded review include the 403 checks, 422 GPS validation, self-verification block, ranked risk list, top reasons, overdue aging bucket, reset duration and an end-to-end path with no manual database edits. [Senior architect review, acceptance tests]

20. Demo Script and Strongest USPs

The demo should show three capabilities that are hard to fake with a static CRUD dashboard.

USP	Live judge-visible sequence	Why it matters
1. Closed-loop accountability	Inspector creates finding -> manager assigns owner -> evidence uploaded -> different user verifies -> action closes -> timeline shows actors and elapsed days	Demonstrates governance as a controlled process, not a reporting screen
2. Explainable configurable risk index	Enterprise dashboard ranks mines -> click high-risk mine -> show component scores and reasons -> adjust a weight -> ranking recalculates	Shows prioritization and explainability without claiming unsupported predictive accuracy
3. Offline field evidence bundle	Turn off network -> submit inspection with GPS and photo -> show pending queue -> reconnect -> server record appears -> map marker appears	Demonstrates a field constraint and a complete evidence path

Five to ten minute demo order: login as Corporate Manager -> show mine ranking -> inspect reasons -> switch to Inspector on phone -> create High-severity observation -> turn network off and save -> reconnect -> create corrective action -> assign owner -> upload evidence -> verify as Mine Manager -> open accountability timeline -> return to dashboard and show updated counts.

The uploaded review rates these same three USPs as the strongest because they directly address accountability, risk prioritization and field evidence reliability while remaining implementable with synthetic data. [Senior architect review, strongest USP recommendation]

21. Production Evolution and Explicit Exclusions

After the hackathon, the modular monolith can evolve without changing the product model. Keep the domain boundaries stable even if deployment architecture changes.

Prototype	Production evolution
JWT + three roles	Enterprise OIDC/SSO, MFA, policy-based authorization
PostgreSQL single instance	Managed HA PostgreSQL, read replicas and backups
Local Docker Compose	Managed container platform, CI/CD, observability
IndexedDB create-only sync	Offline workflow with stronger attachment lifecycle and device controls
Risk index	Validated model only after collecting labeled incident outcomes and monitoring drift
Object storage + SHA-256	Managed encrypted object storage, malware scanning, retention rules
Notification records	Enterprise messaging and escalation integrations
Three demo roles	Full role matrix for regulators, contractors, corporate functions and administrators
Optional OCR	Production document extraction with human review and document provenance

Explicitly do not implement blockchain, custom LLM assistant, IoT ingestion, Kubernetes, zone heatmaps, or a seven-role UI in the hackathon MVP. These features can be revisited only after the core detect-to-close workflow is reliable. The senior review supports these exclusions. [Senior architect review, scope recommendation]

FINAL BUILD ORDER

Schema and deterministic seed -> Auth/RBAC -> Compliance -> Inspection + evidence -> Corrective Action + audit -> Dashboard -> Explainable risk -> PWA offline sync -> GIS/alerts -> testing -> demo freeze.

Appendix A. Practical User Stories and Acceptance Criteria

ID	User story	Acceptance criteria
US-01	As an Inspector, I can record a field observation with GPS and photo so the record has place and evidence.	Cannot submit High/Critical without photo and valid GPS accuracy; successful submission creates server ID.
US-02	As an Inspector, I can save a field observation while offline so reporting does not depend on live connectivity.	Inspection remains visible as pending; reconnect creates exactly one server record.
US-03	As a Mine Manager, I can assign a corrective action with owner and due date.	Action enters ASSIGNED state and audit event identifies manager.
US-04	As a Mine Manager, I can verify evidence and close an action.	Owner cannot self-verify; another authorized manager can close.
US-05	As a Corporate Manager, I can see which mines need attention first.	Dashboard ranks mines by configurable risk index and shows component reasons.
US-06	As a Corporate Manager, I can inspect old overdue work.	Aging buckets let manager isolate actions older than 30 days.
US-07	As a reviewer, I can reconstruct who handled an action.	Timeline shows actor, action, timestamp and elapsed days.
US-08	As the demo operator, I can reset to a known data state.	Deterministic reset completes under 30 seconds and restores seeded ranking.

End of implementation solution. The companion Claude review prompt should be used as a second-pass challenge: it should read this document, identify remaining weak points, propose only implementable USPs, and avoid generic claims.